

Juego Quién es quién en Java

Documentación del trabajo práctico de Programación III

El proyecto implementa un juego de adivinanzas con 23 personajes, turnos y dos máquinas con distintas estrategias. Incluye una interfaz Swing, un modo de observación entre máquinas y un récord local de victorias humanas. Este documento explica las reglas, la organización del código y los algoritmos necesarios para presentar y defender el trabajo.

Funcionamiento general

Cada participante conserva un personaje secreto. En su turno pregunta por una característica o intenta adivinar una identidad. Un acierto gana el duelo. El humano juega primero contra máquina 1; si gana, pasa a máquina 2. Si pierde un duelo, termina el desafío. También se puede observar a las máquinas competir entre sí.

La máquina 1 usa un orden fijo de preguntas. Después de realizar dos preguntas, en cada turno puede arriesgar con 50% de probabilidad o seguir preguntando; si arriesga, elige al azar entre los candidatos restantes, por lo que con dos opciones tiene 50% de probabilidad de acertar. Si falla, descarta esa opción y la partida continúa. La máquina 2 elige divisiones equilibradas mediante una heurística Greedy y aprovecha la información permitida de la primera. Para equilibrar la dificultad, ambas máquinas también pueden cometer un error controlado: repetir una pregunta anterior (30% M1 y 40% M2 cuando ya existe historial). La repetición consume el turno sin descartar candidatos incorrectamente. Las estrategias trabajan con candidatos y nunca reciben el personaje secreto rival.

Ejecución

- Extraer el ZIP completo. En Windows, abrir Ejecutar.bat. El archivo QuienEsQuien.jar ya está compilado y necesita Java 8 o superior.
- Para revisar el código, abrir la carpeta QuienEsQuien en IntelliJ, configurar un JDK y ejecutar el método main de src/app/Main.java.
- Si IntelliJ no reconoce las carpetas, marcar src como Sources Root y pruebas como Test Sources Root.
- Después de modificar fuentes, ejecutar Compilar.bat o sh compilar.sh para actualizar el JAR.
- Las pruebas se ejecutan con Probar.bat o sh probar.sh y requieren un JDK.

Organización de la entrega

src está dividido en packages por responsabilidad: app, modelo, juego, estrategia, persistencia y ui. pruebas contiene el package pruebas con comprobaciones sin JUnit; docs contiene este documento en PDF y Word. README.md reúne las instrucciones de uso. No se requiere Maven, base de datos ni conexión a Internet para jugar.

El código usa clases, herencia, interfaces, enumeraciones, listas, ciclos y condicionales. Swing se ocupa de la interfaz y las clases estándar de archivos guardan el marcador.

Reglas y decisiones del juego

Modo	Participantes y objetivo
Humano por etapas	Etapas: 1: alternan humano y máquina 1. Si gana el humano, se habilita la etapa 2: alternan humano y máquina 2. Hay un tablero humano para cada duelo.
Máquina contra máquina	Turnos: máquina 1 y máquina 2. Cada una intenta descubrir el secreto de la otra. El usuario observa y controla el avance.

- Cada búsqueda comienza con los mismos 23 personajes. Compartir el catálogo no significa compartir la lista mutable de candidatos.
- Los secretos se eligen al crear la partida. Son distintos y no se cambian; el humano conserva su personaje al pasar a la segunda etapa.
- Preguntar o arriesgar consume un turno. Una respuesta filtra únicamente el tablero del objetivo correspondiente.
- Una suposición incorrecta descarta ese personaje de esa búsqueda y pasa el turno. No provoca una derrota inmediata.
- El primer acierto gana el duelo. Vencer a máquina 1 habilita Continuar contra Máquina 2. Vencer a máquina 2 completa el desafío; perder cualquier duelo lo termina.
- No existe un máximo de turnos ni de preguntas. El humano puede repetirlas; una pregunta ya resuelta puede no descartar a nadie.
- Las respuestas del humano se calculan automáticamente desde su ficha fija. Así no se modifica la identidad ni se producen respuestas contradictorias.
- Cada duelo ganado por el humano suma una victoria. Nueva partida abandona el desafío sin sumar otra; las victorias ya guardadas se conservan.

Rasgos y calvicie

Los filtros son género, calvicie, lentes y pelo colorado, negro o amarillo. La calvicie representa pérdida de pelo en la zona superior, conservando cabello lateral y su color. Esta definición permite 24 combinaciones posibles de los cuatro rasgos; el catálogo usa 23 diferentes. Todas pueden distinguirse usando las preguntas disponibles.

Orden inicial e identificadores

Los datos de entrada están agrupados únicamente por género y sus nombres están desordenados. Al cargar cada ficha, el programa asigna el siguiente ID e inserta la ficha en su posición por género y luego nombre ascendente. El ID identifica el alta y no cambia al ordenar; no representa la posición actual de la ficha. Ver ordenamiento muestra las 23 inserciones.

Clases y responsabilidades

La vista recibe acciones y muestra resultados. Partida coordina reglas y turnos; los tableros conservan candidatos y Marcador guarda resultados. Las clases se agrupan además en packages: app (inicio), modelo (entidades y jugadores), juego (reglas y estado), estrategia (Strategy), persistencia (récord) y ui (Swing). Esta separación evita mezclar el algoritmo de búsqueda con la presentación o los archivos.

Clase o grupo	Responsabilidad
app.Main	Crear el catálogo, el marcador y la ventana. Es el punto de entrada.
modelo: Personaje, Genero y ColorPelo	Representar una ficha y los valores permitidos de sus rasgos.
modelo.Pregunta	Definir las siete preguntas y comprobar si una ficha cumple un rasgo.
juego: CatalogoPersonajes y Tablero	Cargar y ordenar fichas; mantener una lista independiente por búsqueda y aplicar filtros.
modelo: Jugador, Humano y Maquina	Guardar el secreto y responder consultas. Humano tiene un tablero por etapa; Maquina tiene uno y delega decisiones en una estrategia.
estrategia: EstrategiaBusqueda y sus implementaciones	Contrato común y dos formas de elegir preguntas: Basica y Greedy.
juego.Partida	Elegir secretos distintos, alternar jugadores, coordinar el arriesgo probabilístico de M1, cambiar de etapa, compartir pistas y determinar victorias.
persistencia: Marcador y Puntaje	Leer y guardar victorias; ordenar el récord.
ui: VentanaJuego e IconoPersonaje	Mostrar controles, fichas y resumen. Ver detalle abre los cálculos completos. Swing dibuja los retratos.

Patrón Strategy

Maquina contiene una referencia de tipo EstrategiaBusqueda. La interfaz ofrece decidirSiAdivinar, elegirPregunta, seleccionarPersonajeSospechoso y getExplicacion. Al crear cada máquina se le asigna una implementación diferente. Maquina usa la misma interfaz en ambos casos: el comportamiento cambia por polimorfismo.

Aplicación de SOLID

- S: cada clase concentra una responsabilidad principal, como filtrar, coordinar turnos, mostrar o persistir.
- O: otra estrategia puede implementarse sin reescribir las decisiones delegadas por Maquina.
- L: Humano y Maquina respetan las mismas operaciones de respuesta definidas por Jugador.
- I: la interfaz de estrategia contiene solo operaciones de decisión y explicación; no exige manejar ventanas o archivos.

- D: Maquina depende de la abstracción EstrategiaBusqueda. Partida crea las implementaciones al preparar el juego.

Inserción ordenada y filtrado

Alta por inserción

CatalogoPersonajes.agregar conserva la lista ordenada después de cada alta. Recorre la lista hasta encontrar el lugar del nuevo personaje y lo inserta allí. ArrayList desplaza las posiciones siguientes. El ID aumenta una vez por alta y permanece fijo.

```
ALTA(nombre, rasgos)
    nuevo = crear personaje con siguienteId
    siguienteId = siguienteId + 1
    posicion = 0
    mientras posicion < cantidad y lista[posicion] va antes que nuevo
        posicion = posicion + 1
    insertar nuevo en posicion
```

Ejemplo: se carga Lucia con ID 1. Luego llega Ana con ID 2 y queda antes de Lucia, porque ambas son del mismo género y Ana va primero por nombre. Las posiciones cambian; los identificadores siguen siendo 2 para Ana y 1 para Lucia.

Filtro lineal de candidatos

Tablero.filtrar recorre únicamente los candidatos actuales. Conserva a quienes producirían la misma respuesta que el rival. Al crear una lista nueva se evita eliminar elementos mientras se recorre la lista original.

```
FILTRAR(candidatos, pregunta, respuesta)
    restantes = lista vacia
    para cada personaje de candidatos
        si pregunta.coincide(personaje) == respuesta
            agregar personaje a restantes
    candidatos = restantes
```

Ejemplo con el catálogo completo: hay 12 personajes femeninos y 11 masculinos. Si la respuesta a “¿Es de género femenino?” es sí, quedan 12; si es no, quedan 11. Volver a preguntar lo mismo no altera un tablero que ya fue filtrado por esa respuesta.

Conservación de la solución

Si las respuestas son correctas, el verdadero secreto siempre cumple el filtro y nunca se descarta. Cada tablero parte del catálogo completo y conserva esa propiedad después de cada pregunta. Un intento fallido elimina solo el ID que se acaba de comprobar.

Cada constructor de Tablero copia la lista recibida. getCandidatos también devuelve una copia. Los objetos Personaje pueden compartirse porque sus atributos son privados y final, y no tienen setters.

Decisiones de las máquinas

Máquina 1 con estrategia básica

Revisa en este orden: pelo colorado, negro, amarillo, calvicie, lentes, femenino y masculino. Elige la primera pregunta que separe a los candidatos en dos grupos no vacíos. Durante sus dos primeras preguntas no arriesga de forma anticipada. A partir de la tercera oportunidad de actuar, si ya realizó al menos dos preguntas y quedan varios candidatos, tiene 50% de probabilidad de arriesgar y 50% de seguir preguntando. El personaje arriesgado se elige al azar entre los candidatos: con 8 opciones la probabilidad de acierto es 1/8; con 2 opciones es 1/2 (50%). Si falla, se elimina esa opción y el juego continúa normalmente. Con un único candidato arriesga con certeza.

Máquina 2 con estrategia Greedy

Evalúa las siete preguntas sobre su lista actual. Para cada una cuenta cuántos candidatos responderían sí y cuántos no. Descarta preguntas que no separan opciones y elige la menor diferencia entre ambos grupos. Si varias empatan, prefiere una conocida por máquina 1; los demás empates respetan el orden del enum Pregunta.

```
mejor = ninguna
menorDiferencia = infinito
para cada pregunta
    si = cantidad de candidatos que cumplen la pregunta
    no = cantidadTotal - si
    si si > 0 y no > 0
        diferencia = valorAbsoluto(si - no)
        si diferencia mejora, o empata y tiene prioridad por historial
            mejor = pregunta
            menorDiferencia = diferencia
retornar mejor
```

Con 23 candidatos, una división 12 y 11 tiene diferencia 1; una división 8 y 15 tiene diferencia 7. La primera deja como máximo 12 posibilidades, cualquiera sea la respuesta, y se prefiere a la segunda. La decisión es local: no se construye ni se explora un árbol completo de partidas futuras.

Elemento de Greedy	Aplicación al juego
Candidatos para elegir	Las preguntas disponibles, evaluadas sobre las fichas restantes.
Selección	Menor diferencia entre las cantidades de sí y no.
Factibilidad	Ambos grupos deben contener al menos una ficha.
Objetivo local	Minimizar la mayor cantidad que puede quedar tras una respuesta.
Solución	Llegar a una única ficha y realizar el intento de adivinación.

Es una heurística Greedy: mejora la división actual, pero no garantiza el mínimo total de turnos ni ganar todas las partidas. En el catálogo actual espera a tener un candidato para arriesgar. Si un catálogo futuro tuviera rasgos repetidos y ninguna pregunta útil, existe una salida que prueba candidatos uno por uno.

Secreto y conocimiento compartido

Encapsulamiento del personaje elegido

Jugador conserva personajeSecreto como private final. No ofrece getter ni setter. Los demás participantes solo pueden consultar un rasgo o comprobar un intento mediante una respuesta booleana. La estrategia recibe exclusivamente una copia de la lista de candidatos: no recibe al rival, a la partida ni a su secreto.

```
private final Personaje personajeSecreto;

public boolean evaluarPreguntaRival(Pregunta pregunta) {
    return pregunta.coincide(personajeSecreto);
}

public boolean esMiPersonajeSecreto(Personaje intento) {
    return personajeSecreto.getId() == intento.getId();
}
```

Partida actúa como árbitro: elige los secretos de las máquinas entre las fichas todavía disponibles. Esa selección evita repeticiones sin pasarle el secreto humano a la estrategia. El resumen con todas las identidades solo se devuelve al terminar. En pantalla, el selector del secreto se bloquea al comenzar.

Ventaja de máquina 2 contra el humano

Durante la etapa 1, Partida guarda las preguntas de máquina 1 y sus respuestas. Máquina 2 todavía no juega. Al ganar el humano y comenzar la etapa 2, máquina 2 recibe ese historial y filtra sus candidatos antes de actuar. Esto es válido porque busca al mismo humano, cuyo personaje no cambia. El humano empieza un tablero nuevo para investigar a máquina 2.

Ejemplo: el humano eligio Ana.
M1 pregunta por pelo colorado: respuesta Si.
M1 pasa de 23 candidatos a 8.
El humano gana y comienza la etapa 2.
M2 recibe la pista guardada: pasa de 23 a 8.
En su turno M2 evalua preguntas sobre esos 8.

Información en máquina contra máquina

La pregunta de máquina 1 se refiere al secreto de máquina 2. La máquina 2 busca un secreto diferente: el de máquina 1. Aplicar esa respuesta a su propia búsqueda sería incorrecto. Por eso conoce la pregunta como antecedente y la prioriza en empates, pero no importa su respuesta. En este modo la diferencia principal de juego proviene de la estrategia Greedy.

El resumen muestra dos líneas por jugada: pregunta y respuesta, más la cantidad restante. Ver detalle permite consultar las listas completas, los conteos, la elección y el historial heredado. Así se puede seguir una partida rápidamente y revisar los procesos cuando se necesiten. El árbitro contesta; la estrategia no hace consultas ocultas al secreto.

Complejidad y alternativas

Se usa n para la cantidad de candidatos, q para la cantidad de preguntas y r para usuarios del récord. En este trabajo n comienza en 23 y q es 7. El análisis explica cómo crecería el trabajo al aumentar los datos; no mide milisegundos ni incluye el dibujo de la interfaz.

Operación	Tiempo	Motivo
Insertar una ficha	$O(n)$	Buscar su posición y desplazar elementos de ArrayList.
Cargar todo el catálogo	$O(n^2)$	Se realizan n inserciones y se registra cada estado de la lista.
Aplicar un filtro	$O(n)$	Se evalúa cada candidato una vez. Usa $O(n)$ de memoria auxiliar.
Buscar o descartar un ID	$O(n)$	Recorrido secuencial; remove puede desplazar elementos.
Elegir pregunta básica o Greedy	$O(qn)$	Para cada pregunta considerada se cuentan coincidencias. Con $q = 7$ resulta $O(n)$.
Ordenar y validar el récord	$O(r^2)$	Inserción y búsqueda de nombres duplicados al cargar. Guardarlo en texto es $O(r)$.

La respuesta a un solo rasgo de una ficha es $O(1)$. Tener preguntas parecidas a una división por mitades no hace que todo el turno sea $O(\log n)$: aquí se recorre la lista para evaluar y filtrar. Las copias de candidatos también cuestan $O(n)$.

Algoritmos no utilizados

Técnica	Justificación
Búsqueda binaria	Necesita una clave ordenada y una comparación que indique qué mitad descartar. Género, lentes y pelo no forman un único orden compatible. Ordenar por nombre no vuelve binarios esos filtros.
Divide y Conquista recursivo	El juego conserva el subconjunto compatible después de una respuesta. No resuelve recursivamente varios subproblemas ni combina sus soluciones. Dividir candidatos no basta para afirmar que se implementó todo el esquema.
Merge Sort o Quick Sort	Son útiles para ordenar grandes colecciones. Con 23 altas y la necesidad de mantener el orden en cada inserción, el recorrido simple es suficiente y fácil de seguir.
Programación dinámica o backtracking	Explorar y almacenar futuros estados o volver atrás complicaría el proyecto. No se busca construir el árbol global óptimo de preguntas.
Dijkstra, Prim o Kruskal	Resuelven problemas sobre grafos, como caminos o conexiones de costo mínimo. Las mecánicas del juego no plantean ese problema.

El historial visible crece con los turnos y no tiene un corte lógico. Como todo programa, su duración práctica depende de la memoria disponible. Las estrategias base evitan preguntas sin efecto, pero la partida puede hacer que una máquina repita deliberadamente una pregunta ya realizada para reducir su ventaja frente al humano. La repetición no altera candidatos y solo consume el turno.

Uso de la interfaz y persistencia

La pantalla presenta las fichas a la izquierda y el historial a la derecha. El selector superior del tablero permite revisar búsquedas independientes. Visualmente las fichas se muestran por ID del 1 al 23. Las fichas rojas están descartadas; las claras siguen siendo posibles.

¿QUIÉN ES QUIÉN?

23 personajes · 2 estrategias · cada pregunta cuenta

Modo

Humano por etapas

Nombre

Tu secreto

2 - Ana

Comenzar

Nueva partida

Primero contra Máquina 1. Ganá para pasar a Máquina 2.
Género, calvicie parcial, lentes y tres colores de pelo. Sin límite de preguntas.

Catálogo completo · ordenado por género y nombre

23 / 23 candidatos

RESUMEN DE LA PARTIDA

2 - Ana Femenino · Colorado Calvicie: No Lentes: Sí	6 - Beatriz Femenino · Negro Calvicie: No Lentes: Sí	4 - Carla Femenino · Colorado Calvicie: Sí Lentes: Sí	8 - Diana Femenino · Negro Calvicie: Sí Lentes: Sí
10 - Elena Femenino · Amarillo Calvicie: No Lentes: Sí	12 - Florencia Femenino · Amarillo Calvicie: Sí Lentes: Sí	9 - Gabriela Femenino · Amarillo Calvicie: No Lentes: No	11 - Helena Femenino · Amarillo Calvicie: Sí Lentes: No
5 - Ines Femenino · Negro Calvicie: No Lentes: No	3 - Julia Femenino · Colorado Calvicie: No Lentes: No	7 - Laura Femenino · Negro Calvicie: No Lentes: No	1 - Lucía Femenino · Colorado Calvicie: No Lentes: No
18 - Andres Masculino · Negro Calvicie: No Lentes: Sí	14 - Bruno Masculino · Colorado Calvicie: No Lentes: Sí	16 - Diego Masculino · Colorado Calvicie: Sí Lentes: Sí	20 - Esteban Masculino · Negro Calvicie: Sí Lentes: Sí
22 - Hugo Masculino · Amarillo Calvicie: No Lentes: Sí	23 - Mateo Masculino · Colorado Calvicie: Sí Lentes: No	21 - Nicolas Masculino · Amarillo Calvicie: No Lentes: No	19 - Pablo Masculino · Negro Calvicie: Sí Lentes: No
17 - Rafael Masculino · Negro Calvicie: No Lentes: No	15 - Sergio Masculino · Colorado Calvicie: Sí Lentes: No	13 - Tomas Masculino · Colorado Calvicie: No Lentes: No	

Gris = descartado. Tocá una ficha para seleccionarla.

Tu jugada

¿Es de género femenino?

Preguntar

Mi sospechoso

Arriesgar personaje

Siguiente turno

Reproducir

Récords

Ver ordenamiento

Ver detalle

CÓMO JUGAR

Primero competís contra Máquina 1. Si ganás, pasás a Máquina 2.

Tu personaje se mantiene entre etapas y Máquina 2 recuerda las preguntas de la primera.

Preguntá o arriesgá en tu turno. Cada duelo ganado suma una victoria al récord.

En máquina contra máquina, usá Siguiente turno o Reproducir.

El resumen muestra las jugadas. Ver detalle muestra los cálculos de las máquinas.

Pantalla inicial con las 23 fichas. Los retratos representan los mismos rasgos usados por las preguntas.

Recorrido de una partida

- Elegir modo, escribir el nombre y seleccionar el personaje humano con el desplegable o una ficha. Presionar Comenzar.
- Preguntar o arriesgar contra máquina 1. Después actúa esa máquina. Si ganás, presionar Continuar contra Máquina 2; tu personaje permanece fijo.
- En máquina contra máquina, usar Siguiente turno o Reproducir/Pausar. El resumen tiene dos líneas por jugada; Ver detalle pausa y abre los cálculos completos.
- Al finalizar el desafío se revelan los secretos. Nueva partida permite elegir de nuevo. Ver ordenamiento muestra las altas paso a paso.

Archivo del récord

Marcador guarda nombre y victorias en un CSV UTF-8 en la carpeta QuienEsQuien del usuario. La ventana Récords muestra dos columnas: NOMBRE y VICTORIAS, con cada número debajo de su encabezado, además de la ruta del archivo. Cada duelo humano ganado suma una victoria: hasta dos por desafío. Perder después o abandonar conserva la primera. Las simulaciones de máquinas no suman.

La carga valida el formato. Si el archivo está dañado, se informa el problema y no se reemplaza con datos vacíos. La escritura usa primero un archivo temporal. El récord está pensado para una sola instancia del juego a la vez.

Catálogo de personajes

Esta es la lista resultante en orden por género y nombre. El ID corresponde al orden de alta, por eso no aparece necesariamente de menor a mayor. Cada fila tiene una combinación distinta de rasgos.

ID	Nombre	Género	Calvicie	Lentes	Pelo
2	Ana	Femenino	No	Sí	Colorado
6	Beatriz	Femenino	No	Sí	Negro
4	Carla	Femenino	Sí	Sí	Colorado
8	Diana	Femenino	Sí	Sí	Negro
10	Elena	Femenino	No	Sí	Amarillo
12	Florencia	Femenino	Sí	Sí	Amarillo
9	Gabriela	Femenino	No	No	Amarillo
11	Helena	Femenino	Sí	No	Amarillo
5	Ines	Femenino	No	No	Negro
3	Julia	Femenino	Sí	No	Colorado
7	Laura	Femenino	Sí	No	Negro
1	Lucia	Femenino	No	No	Colorado
18	Andres	Masculino	No	Sí	Negro
14	Bruno	Masculino	No	Sí	Colorado
16	Diego	Masculino	Sí	Sí	Colorado
20	Esteban	Masculino	Sí	Sí	Negro
22	Hugo	Masculino	No	Sí	Amarillo
23	Mateo	Masculino	Sí	No	Amarillo
21	Nicolas	Masculino	No	No	Amarillo
19	Pablo	Masculino	Sí	No	Negro
17	Rafael	Masculino	No	No	Negro
15	Sergio	Masculino	Sí	No	Colorado
13	Tomas	Masculino	No	No	Colorado

Distribución: 12 personajes femeninos y 11 masculinos; 11 con calvicie y 12 sin ella; 11 con lentes y 12 sin lentes. Hay 8 de pelo colorado, 8 de pelo negro y 7 de pelo amarillo.

Con dos valores de género, dos de calvicie, dos de lentes y tres colores existen 24 combinaciones. Se omite la combinación masculino, con calvicie, con lentes y pelo amarillo para conservar exactamente 23 fichas.

Cambios respecto de la base recibida

Se conservaron las ideas de Jugador, Maquina y EstrategiaBusqueda. Se corrigió el nombre del archivo de la interfaz y se quitaron imports inválidos. Jugador pasó a encapsular el secreto sin getter; los tableros usan copias. Maquina es concreta y delega en Strategy. El código quedó organizado en packages por responsabilidad. Las decisiones humanas se toman desde la vista y la espera se maneja con Timer de Swing, sin detener la ventana con sleep.

Cumplimiento y comprobaciones

Consigna	Implementación y comprobación
Java e interfaz visual	Aplicación Swing con Main, controles, fichas y registro de procesos.
23 opciones iguales	Catálogo de 23 combinaciones únicas; cada tablero empieza con una copia completa.
Carga incremental ordenada	Entrada agrupada por género, ID autoincremental e inserción por género y nombre. Registro accesible desde la interfaz.
Secretos diferentes y fijos	Selección sin reemplazo, campo private final, ausencia de setter y selector bloqueado.
Máquina sin acceso al secreto humano	Strategy solo recibe candidatos. Jugador responde con booleanos y no expone la identidad.
Preguntar o arriesgar por turnos	Solo alternan los dos jugadores del duelo actual. Ganar la etapa 1 habilita la segunda; perder termina el desafío.
Dos máquinas y comportamientos distintos	M1 usa orden fijo y, tras 2 preguntas, puede arriesgar con 50% de probabilidad; M2 usa Greedy y hereda las pistas de la primera etapa al comenzar su duelo contra el mismo humano.
Preguntas sin máximo y derrota por acierto rival	Sin contador límite. Se permiten repeticiones; fallar consume turno. Un acierto gana el duelo.
Récord con usuario y victorias	CSV persistente, una victoria por duelo ganado. Compara nombres sin distinguir mayúsculas ni espacios externos.
Documentación de patrones y algoritmos	Strategy, principios SOLID, inserción, filtro, Greedy, pseudocódigo, complejidades y alternativas justificadas.

Verificaciones ejecutadas

El proyecto se compiló para Java 8 (class version 52) y las pruebas se ejecutaron con Java 21. PruebasJuego superó 8613 comprobaciones: los 23 secretos, filtros, estrategias, arriesgo anticipado de Máquina 1 después de dos preguntas, 200 simulaciones entre máquinas, turnos por etapa, transición, historial heredado, derrotas, resumen breve y persistencia.

La interfaz se renderizó y se probaron sus controles: ambos modos, secreto fijo, avance a máquina 2 solo tras ganar y guardado de ambas victorias sin duplicarlas al refrescar. Las pruebas se ejecutaron en Linux sin pantalla física; no se utilizó Windows.

Resultado al resolver los 23 secretos por separado

Estrategia	Turnos acumulados	Intentos incorrectos
Básica aislada	129	0
Greedy aislada	129	0

Esta comparación aislada mide únicamente las implementaciones de EstrategiaBasica y EstrategiaGreedy. En esa prueba ambas esperan a aislar una ficha; el arriesgo probabilístico de Máquina 1 pertenece a la coordinación de Partida y se prueba por separado. En el juego real, M1 puede intentar acertar antes con varios candidatos, mientras Greedy normalmente espera certeza.

Preguntas para la defensa

¿Qué hace el proyecto?

Permite investigar personajes por rasgos. El humano juega contra máquina 1 y, si gana, contra máquina 2. También se puede observar un duelo entre las máquinas.

¿Por qué una interfaz de estrategia?

Para cambiar cómo decide una máquina sin reescribir la clase Maquina. Las dos implementaciones cumplen el mismo contrato.

¿Por qué el secreto es private final?

Private impide el acceso directo desde otras clases. Final evita reemplazarlo después del constructor. Además, no existe un getter del secreto.

¿La máquina hace preguntas ocultas?

No. Cuenta rasgos de las fichas públicas para elegir una pregunta. La respuesta sobre el secreto se solicita únicamente cuando ejecuta el turno.

¿Cómo se filtra?

Se recorre la lista actual y se copian a otra lista los personajes cuya respuesta coincide con la recibida.

¿Qué decisión Greedy se toma?

Elegir la pregunta que deja los grupos de sí y no más equilibrados. Es una decisión local; no se evalúan todos los futuros posibles.

¿Greedy garantiza ganar?

No. Evita arriesgar antes de identificar una ficha en este catálogo, pero una estrategia que arriesga puede acertar antes.

¿Cuál es la ventaja de máquina 2?

Recibe las preguntas y respuestas guardadas en la primera etapa; el humano mantiene su secreto. Entre máquinas solo usa las preguntas para desempatar porque los objetivos son diferentes.

¿Por qué no es búsqueda binaria?

Las preguntas por rasgos no indican izquierda o derecha dentro de una única clave ordenada. Además, se recorren candidatos para contar y filtrar.

¿Por qué usar inserción?

Son solo 23 fichas y hay que mantenerlas ordenadas durante la carga. Es fácil mostrar dónde entra cada una; cargar todas cuesta $O(n^2)$.

¿Qué ocurre si repito una pregunta o fallo?

Repetir está permitido y puede dejar la lista igual. Fallar descarta esa ficha y consume el turno. La derrota llega cuando otro acierta.

¿Cómo continúa el récord al cerrar?

Marcador lee el CSV de la carpeta del usuario. Cada duelo humano ganado suma una victoria. Perder la segunda etapa no borra la primera victoria; iniciar otra partida tampoco.

Material de apoyo y recorrido del código

Relación con el material de clase

Archivo de clase	Relación con el trabajo
Programacion_3_SOLID(1).pptx	Diapositivas 3 a 7: responsabilidades, extensión mediante abstracciones, sustitución, interfaces pequeñas y dependencia de abstracciones.
Greedy parte 1.pptx	Diapositivas 2, 3 y 7: elección local, elementos de una solución Greedy y condiciones para hablar de optimalidad. La estrategia del juego es heurística.
2023 02 DyC parte 1.pdf	Página 84 del PDF: ordenamiento por inserción. Se aplica su idea al alta incremental de fichas y al récord.
Notación Big O y analisis de algoritmos.pptx	Análisis de comparaciones y ciclos. Fundamenta los costos lineales y cuadráticos del proyecto.
Divide y conquista con actividad.pptx	Diapositivas 5, 9, 12 y 13: búsqueda por mitades, recursión, Merge Sort y diferencia entre un algoritmo y separar un programa en clases.
Guia de Ejercicios Programación III.pdf	Organización de técnicas: Divide y Conquista, Greedy, grafos, programación dinámica y backtracking. Se explican las que no se necesitan en este juego.

Orden sugerido para estudiar las clases

- Personaje y Pregunta: entender qué datos tiene una ficha y cómo se evalúa cada característica.
- CatalogoPersonajes y Tablero: seguir una inserción, una copia de lista y un filtro.
- Jugador, Humano y Maquina: ubicar el secreto, la herencia y la delegación de decisiones.
- EstrategiaBasica y EstrategiaGreedy: comparar cómo eligen preguntas a partir de la misma lista y distinguir esa decisión del arriesgo probabilístico que coordina Partida.
- Partida: revisar preguntarHumano, jugarTurnoMaquina, la probabilidad de arriesgo de M1, recordarPregunta, iniciarSegundaEtapa y siguienteTurno.
- Marcador: revisar cargar, registrarVictoria y guardar. Explicar por qué se lee antes de sumar.
- VentanaJuego y app.Main: reconocer eventos, controles y el Timer. Las reglas centrales siguen en Partida.

Demostración breve

Abrir Ver ordenamiento, iniciar un duelo de máquinas y avanzar dos turnos. Mostrar el resumen y abrir Ver detalle para comparar las estrategias. En modo humano, explicar que ganar a máquina 1 habilita la etapa 2 y que su historial se hereda sin cambiar el secreto humano. Entre máquinas no se reutilizan respuestas, porque buscan secretos diferentes. Cerrar con el CSV y el campo privado del secreto.